

MADHAV INSTITUTE OF TECHNOLOGY & SCIENCE, GWALIOR(M.P.), INDIA

Deemed University

NAAC ACCREDITED WITH A++ GRADE

Design and Analysis of Algorithms Lab (25252107)

Lab Report

Submitted for the partial fulfillment of the degree of

Bachelor of Technology

In

Mathematics and Computing

Submitted By

Shivam Rawat

BTMC25O1129

Faculty Mentor

Dr. Dheeraj Kumar Dixit

Head of department

Mathematics and computing

January 2026

Index

Sr no.	Program name	date

Experiment 1: Linear Search

Program:

```
int LinearSearch(int arr[], int n, int x){
for(int i=0; i<=n-1; i++){
if (arr[i] == x){
Return i;
}
Return -1;
}}};
```

Output :-

```
Enter the number of elements: 4
Enter the elements: 30
20
12
3
Enter the element to search: 12
Element found at index 2
```

Experiment 2: Binary Search

Program:

```
int BinarySearch(int arr[], int n, int target){

int low = 0;
int high = n-1;

while(low<=high){

mid = (low+high)/2;

if (arr[mid] == target){
return mid;
}

if(arr[mid]<target){
low = mid + 1;
}

If(arr[mid]>target){
High = mid-1;
}

Return -1;
}};
```

Ouput:-

```
Enter number of elements: 5
Enter elements in sorted order: 20
30
40
50
60
Enter element to search: 30
Element found at index 1
```

Experiment 3: Bubble Sort

Program:

```
void bubbleSort(vector<int>& arr) {
int n = arr.size();
bool swapped;

for (int i = 0; i < n - 1; i++) {
for (int j = 0; j < n - i - 1; j++) {
if (arr[j] > arr[j + 1]) {
swap(arr[j], arr[j + 1]);
}
}
break;
}};
```

Output:-

```
Enter number of elements: 5
Enter elements: 12
14
25
58
89
Sorted array: 12 14 25 58 89
```

Experiment 4: Selection Sort

Program:

```
int SelectionSort(int a[], int n)

{

for(int i=0; i<n-1; i++) {

for(int j=i+1; j<n;j++) {

if(a[j]<a[i]) {

swap(a[j],a[i]);

}

else{

continue;

}}}};
```

Output

```
  SelectionSort } ; if ($?) { .\SelectionSort }
● 10 23 45 46 89
```

Experiment 5: Insertion Sort

Program:

```
void insertionSort(int arr[], int n)
{
for (int i = 1; i < n; ++i) {
int key = arr[i];
int j = i - 1;
while (j >= 0 && arr[j] > key) {
arr[j + 1] = arr[j];
j = j - 1;
}
arr[j + 1] = key;
}};
```

Output

```
Enter number of elements: 5
Enter elements: 34
34
56
34
32
Sorted array: 32 34 34 34 56
```

Experiment 6: Quick Sort

```
int partition(int arr[], int low, int high) {

int pivot = arr[high];

int i = low - 1;


for (int j = low; j <= high - 1; j++) {

if (arr[j] < pivot) {

i++;

swap(arr[i], arr[j]);

}}

swap(arr[i + 1], arr[high]);

return i + 1;

}
```

Output

```
Enter number of elements: 5
Enter elements: 39
28
228
38
89
Sorted array: 28 38 39 89 228
```

Experiment 7: Merge Sort

Program:

```
void merge(int arr[], int low, int mid, int high) {
 int i = low; int j = mid + 1;
int k = 0;
int temp[100];
while (i <= mid && j <= high) {
if (arr[i] < arr[j]) {
temp[k] = arr[i]; i++;
}
 else
{
temp[k] = arr[j]; j++;
 }
k++;
}
// Copy remaining elements from left part
while (i <= mid) {
temp[k] = arr[i]; i++; k++;
 }
 // Copy remaining elements from right part
 while (j <= high) {
temp[k] = arr[j]; j++; k++;
}
 // Copy sorted elements back to original array
 for (i = low, k = 0; i <= high; i++, k++) {
arr[i] = temp[k];
 }
 }

// Function for Merge Sort void
mergeSort(int arr[], int low, int high) {
 if (low < high) {
 int mid = (low + high) / 2;
mergeSort(arr, low, mid);
mergeSort(arr, mid + 1, high);
merge(arr, low, mid, high);
} };
```

Output

```
Enter number of elements: 5
Enter elements: 23
67
45
47
27
Sorted array: 23 27 45 47 67
```

Experiment 8: Min Max Algo

Program:

```
   void maxMin(int arr[], int l, int r, int &max, int &min)
   {
   // Only one element
   if (l == r)
   {
   max = min = arr[l];
   }
// Only two elements
   else if (r - l == 1)
   {
     if (arr[l] > arr[r])
      {
        max = arr[l];
        min = arr[r];
       }
     else{
     max = arr[r];
     min = arr[l];
     }}
 // More than two elements
    else {
    int mid = (l + r) / 2;
    int max1, min1;


// Find max and min of left half
    maxMin(arr, l, mid, max, min);

// Find max and min of right half
    maxMin(arr, mid + 1, r, max1, min1);


// Compare maximums
   if (max < max1)
    {
     max = max1;
    }
// Compare minimums
    if (min > min1)
    {
     min = min1;
     }}}
```

Output

```
Enter number of elements: 5
Enter elements: 23
34
56
68
89
Maximum element = 89
Minimum element = 23
```

Experiment 9: Matrix Multiplication

```
int main()
{
    int A[2][2], B[2][2], C[2][2];



    // Input first matrix
    cout << "Enter elements of first 2x2 matrix:" << endl;
    for (int i = 0; i < 2; i++)
    {
        for (int j = 0; j < 2; j++)
        {
            cin >> A[i][j];
        }
    }



    // Input second matrix
    cout << "Enter elements of second 2x2 matrix:" << endl;
    for (int i = 0; i < 2; i++)
    {
        for (int j = 0; j < 2; j++)
        {
            cin >> B[i][j];
        }
    }
```

```
// Strassen's 7 calculations
int P1 = A[0][0] * (B[0][1] - B[1][1]);
int P2 = (A[0][0] + A[0][1]) * B[1][1];
int P3 = (A[1][0] + A[1][1]) * B[0][0];
int P4 = A[1][1] * (B[1][0] - B[0][0]);
int P5 = (A[0][0] + A[1][1]) * (B[0][0] + B[1][1]);
int P6 = (A[0][1] - A[1][1]) * (B[1][0] + B[1][1]);
int P7 = (A[0][0] - A[1][0]) * (B[0][0] + B[0][1]);

// Calculate result matrix
C[0][0] = P5 + P4 - P2 + P6;
C[0][1] = P1 + P2;
C[1][0] = P3 + P4;
C[1][1] = P5 + P1 - P3 - P7;

// Display result
cout << "Resultant Matrix:" << endl;

for (int i = 0; i < 2; i++)
{
    for (int j = 0; j < 2; j++)
    {
```

```
            cout << C[i][j] << " ";
        }
        cout << endl;
    }


    return 0;
}
```

Output

```
Enter elements of first 2x2 matrix:
21
23
45
56
Enter elements of second 2x2 matrix:
12
90
2
34
```

Experiment 10 : Heap sort

```
#include <iostream>

using namespace std;

// Function to heapify a subtree rooted at index i

void heapify(int arr[], int n, int i){

// Assume root is largest

    int largest = i;

// Left child

 int l = 2 * i + 1;

 // Right child

  int r = 2 * i + 2;

   // If left child is larger than root

    if (l < n && arr[l] > arr[largest])    {

   largest = l;

  }  // If right child is larger than largest

   if (r < n && arr[r] > arr[largest]){

   largest = r;

   }

  // If largest is not root

   if (largest != i)  {

    swap(arr[i], arr[largest]);

 // Heapify the affected subtree

       heapify(arr, n, largest);

  }}

// Heap Sort function

void heapSort(int arr[], int n){

   // Build max heap
```

```
   for (int i = n / 2 - 1; i >= 0; i--){
   heapify(arr, n, i);
   }
// Extract elements one by one
   for (int i = n - 1; i > 0; i--) {
 // Move largest element to the end
   swap(arr[0], arr[i]);
 // Heapify the remaining heap
     heapify(arr, i, 0);
 }}
int main(){
int arr[] = {9, 4, 3, 8, 10, 2, 5};
int n = sizeof(arr) / sizeof(arr[0]);
heapSort(arr, n);
cout << "Sorted array: ";
for (int i = 0; i < n; i++){
cout << arr[i] << " ";
 }
return 0;
}
```

Output

```
Sorted array: 2 3 4 5 8 9 10
PS C:\Users\ASUS\Documents\DAA>
```